

1.1 - MORE ON NETWORK TRAINING

CHAPTER INDEX

☐ [LEARNING RATE](#)

- [LEARNING RATE ANNEALING](#)
- [CYCLIC LEARNING RATE](#)
- [MOMENTUM](#)
- [SUPERCONVERGENCE](#)
- [OTHER APPROACHES](#)

☐ [OVERFITTING AND DROPOUT](#)

- [DROPOUT](#)

☐ [DATA AUGMENTATION](#)

- [MOTIVATION, INTRA-CLASS VARIATIONS AND GENERALISATION](#)
- [GEOMETRIC TRANSFORMATIONS](#)
- [LOCAL FILTERS](#)
- [ADDING NOISE](#)
- [OTHER TRANSFORMATIONS](#)

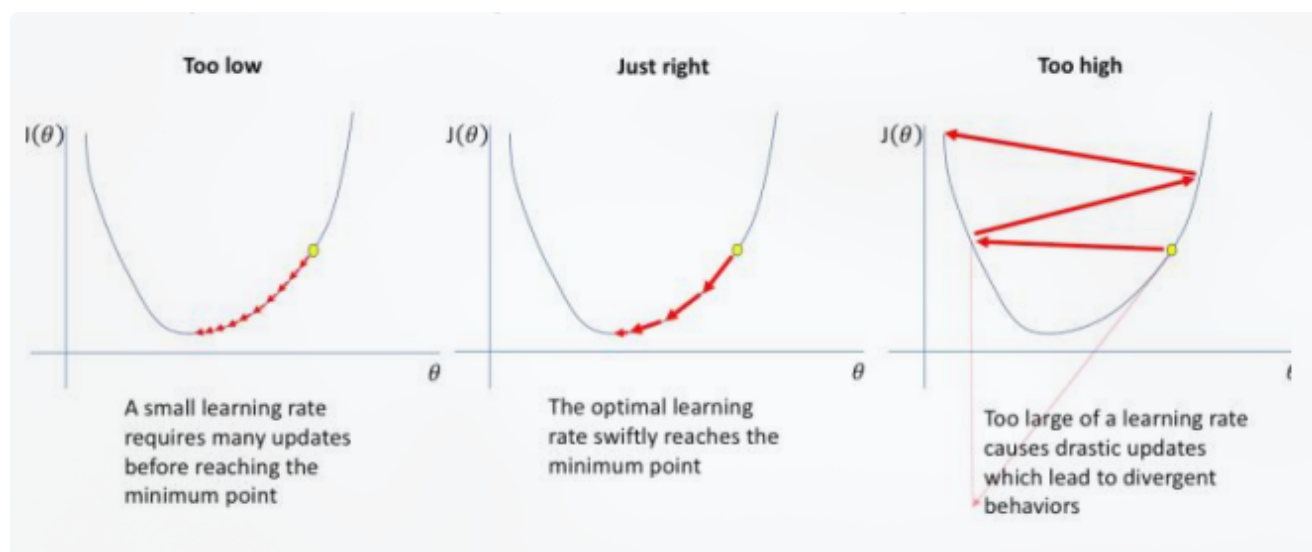
LEARNING RATE

When discussing *gradient descent* we've seen how the **update rule for weights** is:

$$\mathbf{w}_{\text{curr}} = \mathbf{w}_{\text{curr}} - \eta \nabla J(\mathbf{w}_{\text{curr}})$$

And we said that η is the **learning rate**.

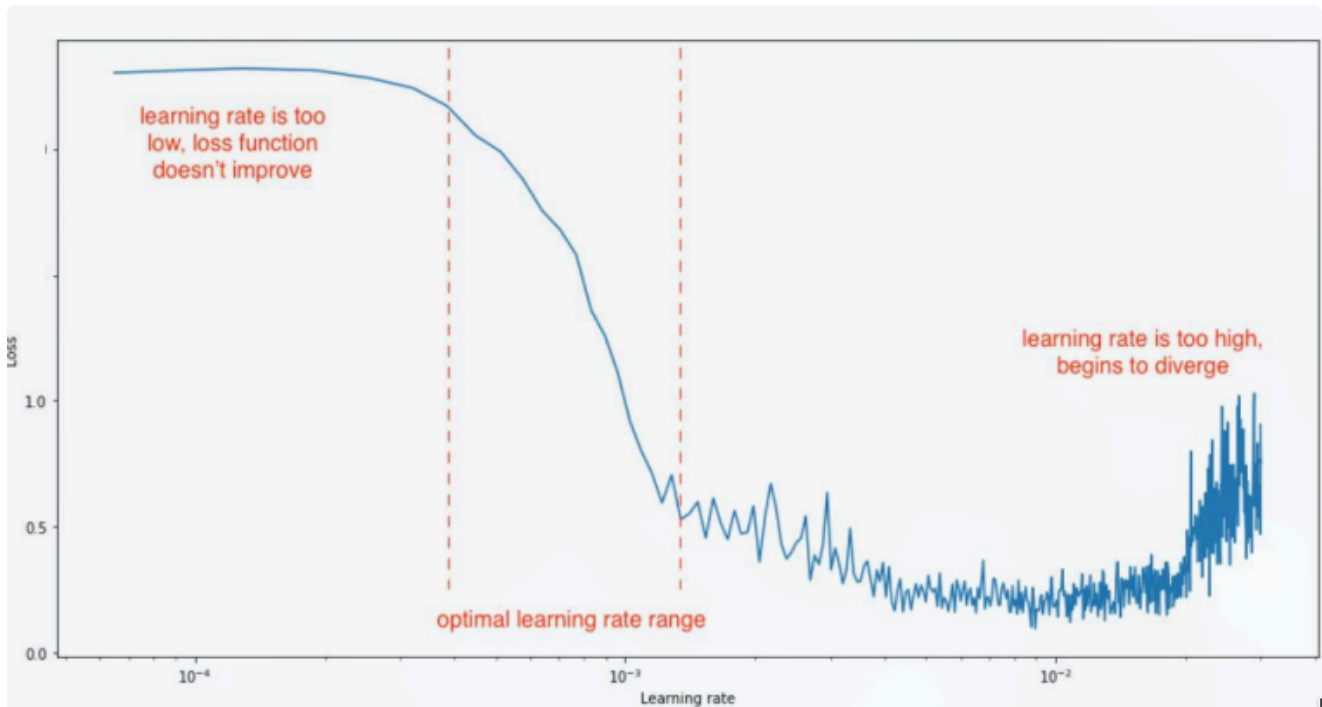
Calibrating the learning rate is crucial: if it's *too low*, **convergence is too slow**; on the contrary, if it's *too large*, the weights **swing excessively** and may even **diverge**.



Notice that the **optimal value** depends on the **network architecture**, the choice of **loss function** and other parameters.

When working with *well known architectures* that have already been documented by

others, it's usually recommended to **start with the suggested values** and then **slightly tweak them** by *monitoring convergence and generalisation*.



KEY NOTE

During the experiment, we should **observe all three phases**, as this makes **choosing the optimal range trivial** (see figure above).

LEARNING RATE ANNEALING

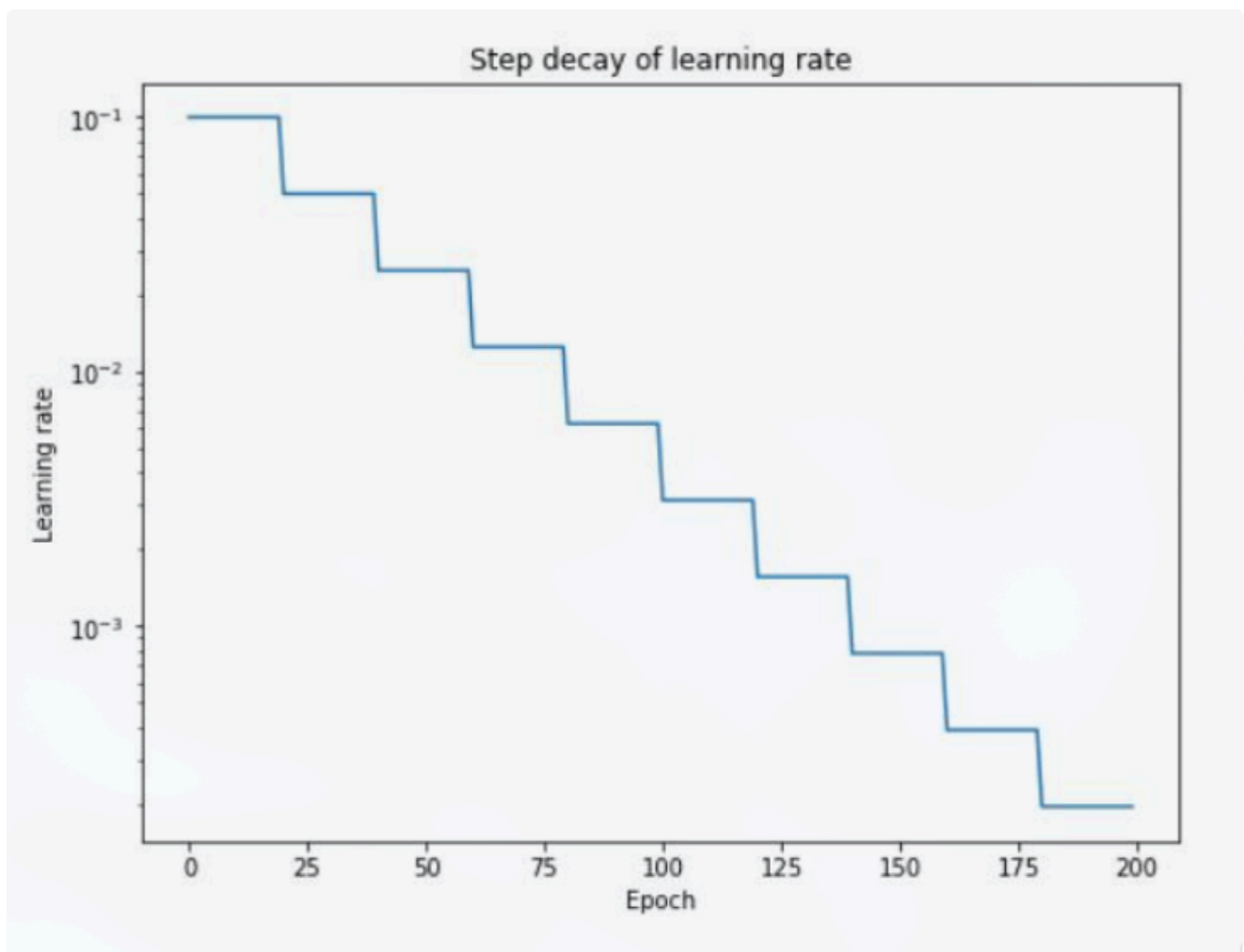
A commonly employed technique to **tune the value** of the learning rate is **learning rate annealing**.

LEARNING RATE ANNEALING

Start with a high learning rate and then **gradually lower it** during training.

The intuition behind this approach is that we would like to **traverse quickly** from the **initial parameters** to a **range of good parameter values**, but then we want a learning rate **small enough** to **explore the deeper but narrower parts** of the loss function.

The most popular form of learning rate annealing is a **step decay** where the learning rate is **reduced by some percentage after a set number of training epochs**.



CYCLIC LEARNING RATE

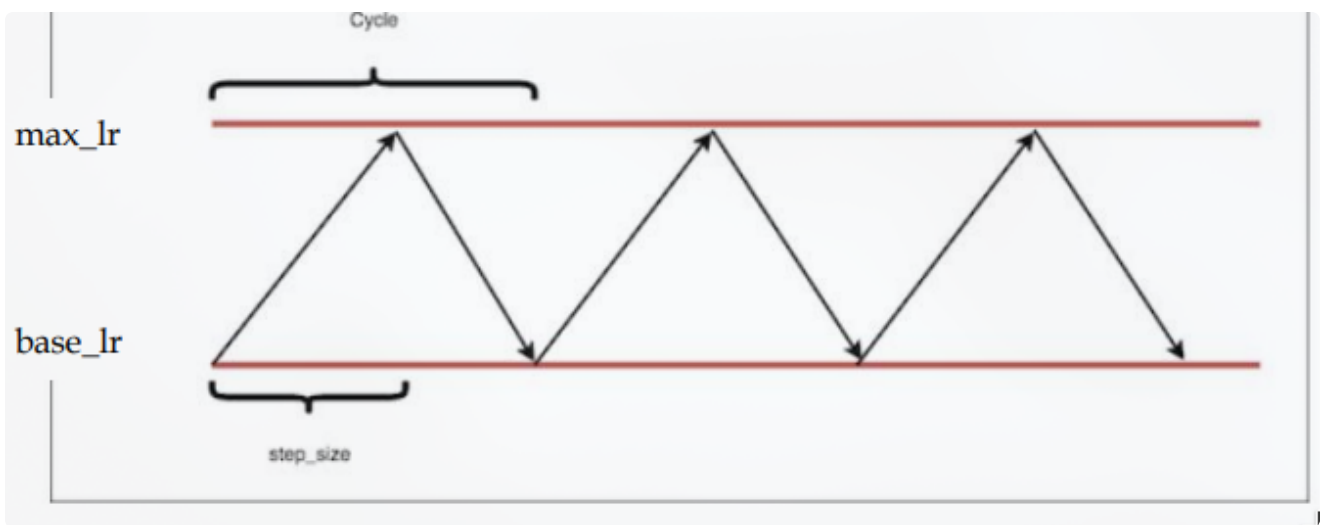
More generally, we can establish that is useful to define a **learning rate schedule** according to which the learning rate is updated during training with some **specified rule**.

📋 CLR

One possible solution is **cyclic learning rate**, which consists in changing the learning rate in a **cyclical way**, between **two bounds**.

Thus, it is important to define two things:

1. The **bounds** between which the LR varies.
2. The **step size**, that is, how many epochs will it take to *go from bound to another*.

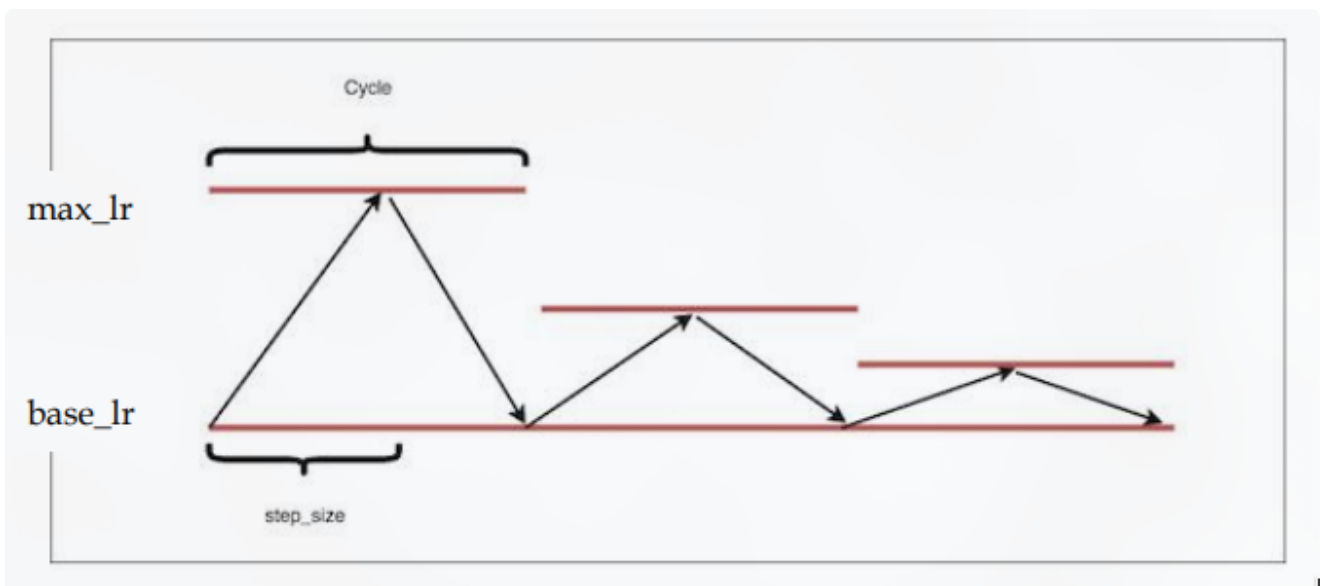


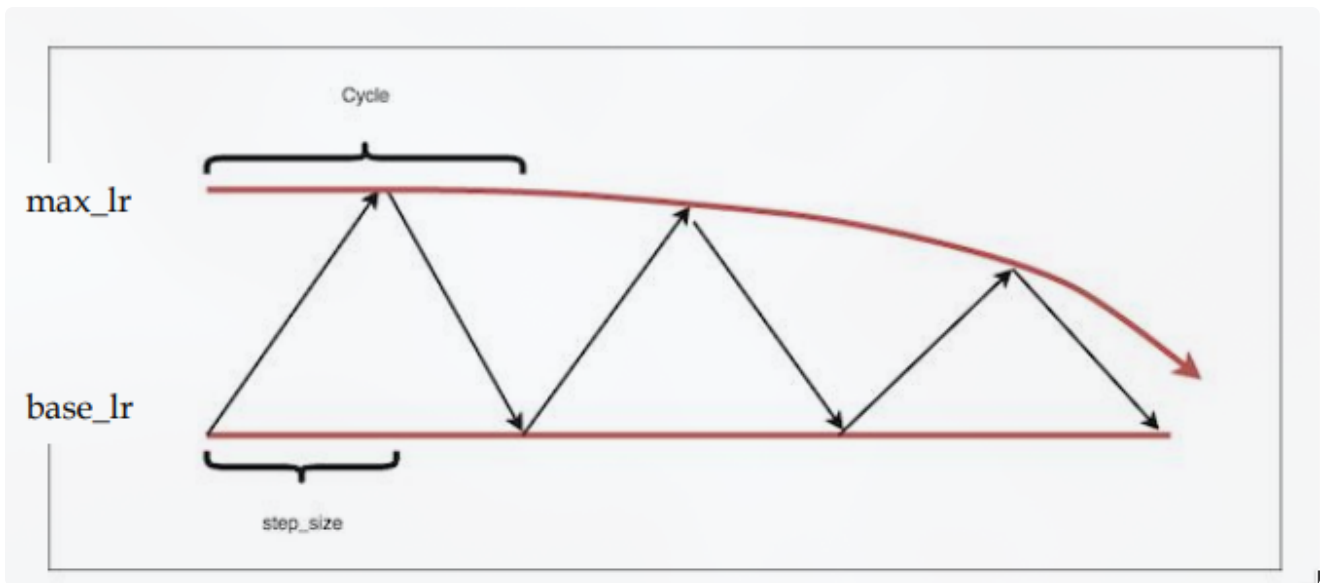
💡 INTUITION

A network with **good generalisation properties** should be robust in the sense that **small changes to the parameters shouldn't cause drastic changes in performance**.

Thus, we want to **avoid sharp local minima** and, by allowing our **learning rate to increase periodically**, we achieve exactly this: *we jump out of these minima*, temporarily increasing the loss, but ultimately **leading to convergence on a more desirable minimum**.

Notice that, other than the triangular profile presented above, other forms of CLR are possible:





In the first case, `max_lr` is **halved after every cycle**, while in the second case it is **reduced exponentially** with each iteration.

There are **no mathematical rules** to set `max_lr` and `min_lr`. The suggested method is to **check past papers** related to the problem that is being analysed.

In any case, the LR schedule can be written as follows:

$$\eta_t = \eta_{\min} + (\eta_{\max} - \eta_{\min})(\max(0, 1 - x))$$

With the following parameters:

$$x = \left| \frac{i}{s} - 2c + 1 \right|$$

$$c = \left\lfloor 1 + \frac{i}{2s} \right\rfloor$$

Where by i is the current **iteration** (that is, how many *times we have performed a weight update*) and s is the **step size**, or half the cycle length.

NOTE

Notice that $\eta_{\max}$ can be a **decreasing function**, as seen in the figures above.

OBSERVATION

Notice that c is counting **which cycle we are in**.

The quantity $\frac{i}{s} - 2c$ counts **how many half cycles are left in the current cycle** (thus, this value is in $[-2, 0]$ and adding 1 shifts it to $[-1, 1]$).

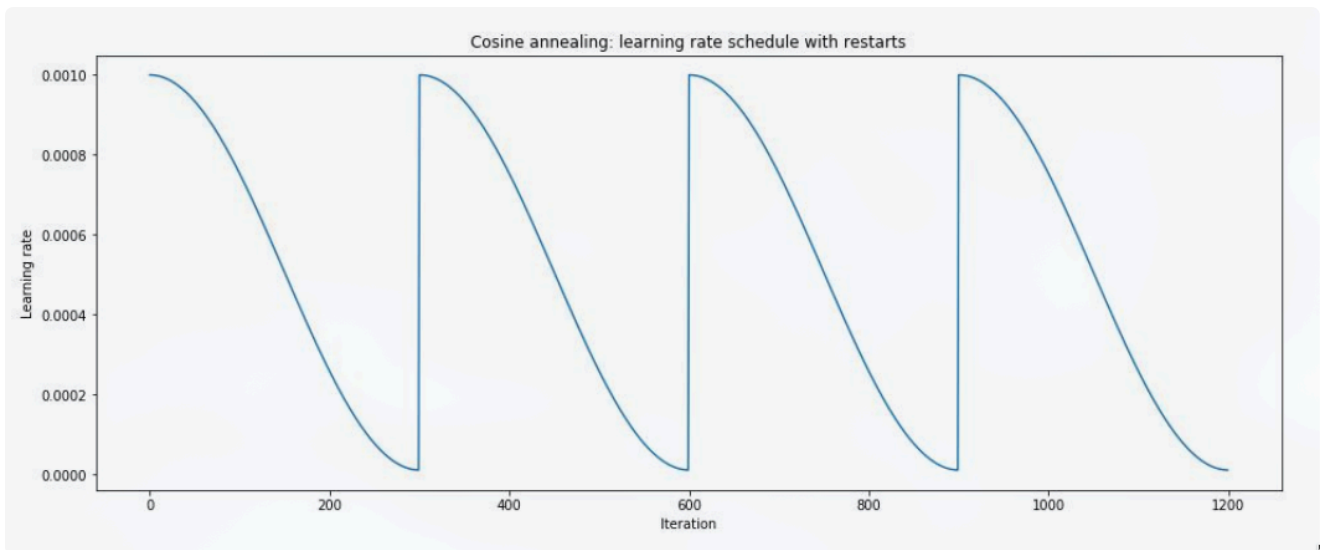
Finally, by applying the **absolute value** we obtain a **triangular shape for each cycle**.

We consider $1 - x$ so that the **schedule starts at the minimum value** and we multiply by $\eta_{\max} - \eta_{\min}$ to **match the desired interval**.

Furthermore, there exist **other cyclic patterns** that don't rely on triangular shapes. Still, they are computed in very similar ways: below, **cosine annealing with warm restarts** is presented:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{T_{\text{curr}}}{T_i} \pi \right) \right)$$

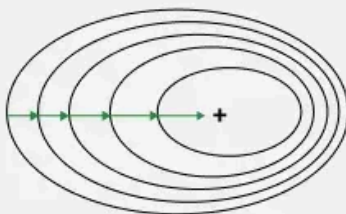
Where T_i is the **number of epochs per cycle** and T_{curr} is the **number of epochs since the last restart**. The profile ends up looking as shown below:



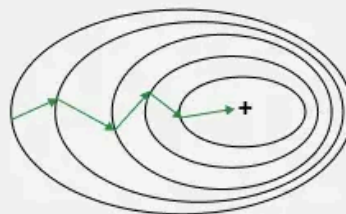
MOMENTUM

We've seen that SGD with **mini-batches** is a **compromise between noise and stability**, but it still causes a **"zig-zag" movement** of the gradient, which **slows convergence down** and makes the process **less stable**.

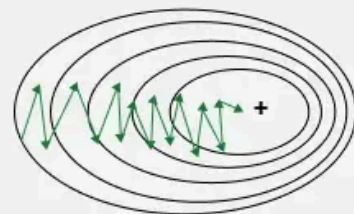
Different Variants of Gradient Descent



Batch Gradient Descent

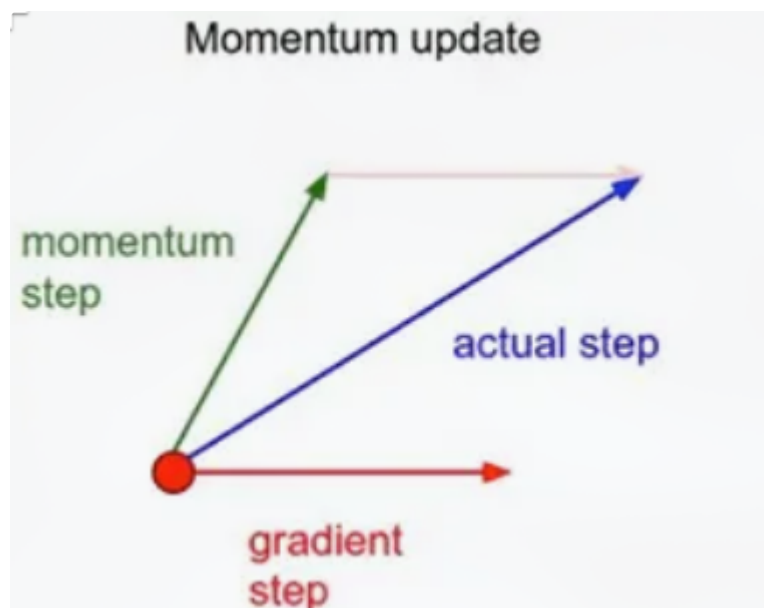


Mini-Batch
Gradient Descent



Stochastic
Gradient Descent

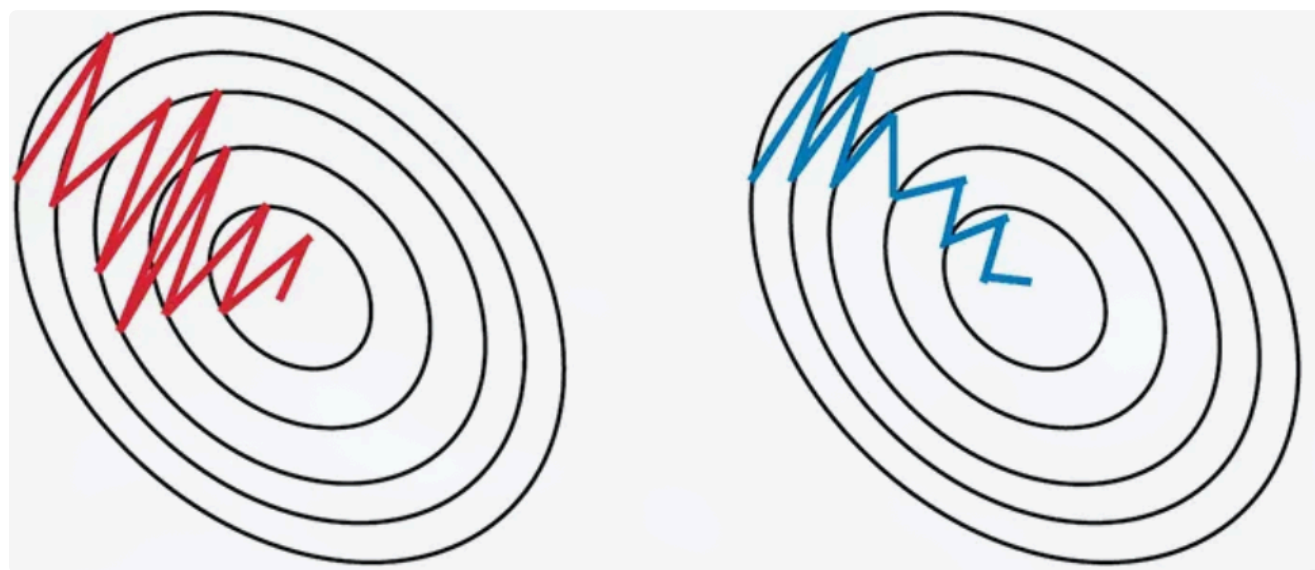
To address this problem, we can **add momentum to the delta value** applied to the **weights**: this added term **depends on the previous update** and has the effect of **preserving part of the previous "motion"**, thus **stabilising the path taken by the gradient** (just like momentum in physics).



When momentum is considered, weight updates are calculated as follows:

$$\Delta w_{ij}^k = \eta \frac{\partial J}{\partial w_{ij}} + \underbrace{\gamma \Delta w_{ij}^{k-1}}_{\text{momentum}}$$

Where Δw_{ij}^{k-1} is the **previous weight update**.



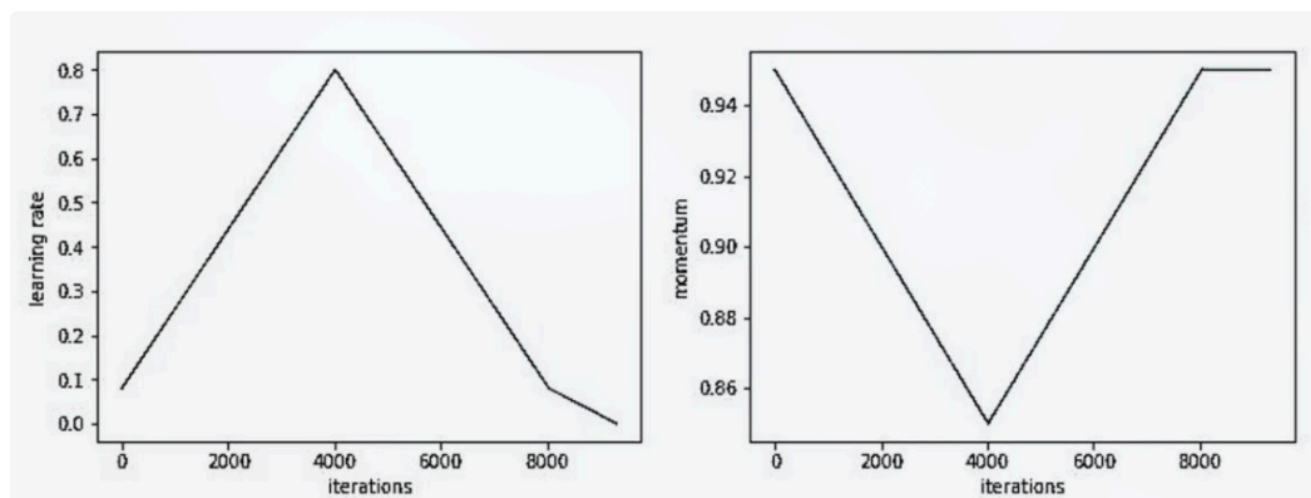
In the figure above we see SGD without momentum on the left, and **SGD with momentum** on the right: applying momentum *adds stability*.

SUPERCONVERGENCE

Both **cyclic learning rate** and **momentum** should be used when **training** a neural network.

These days, the **most performant** general-purpose *learning rate scheduler* is the **one-cycle learning rate scheduler**, introduced in the 2017 paper "**Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates**".

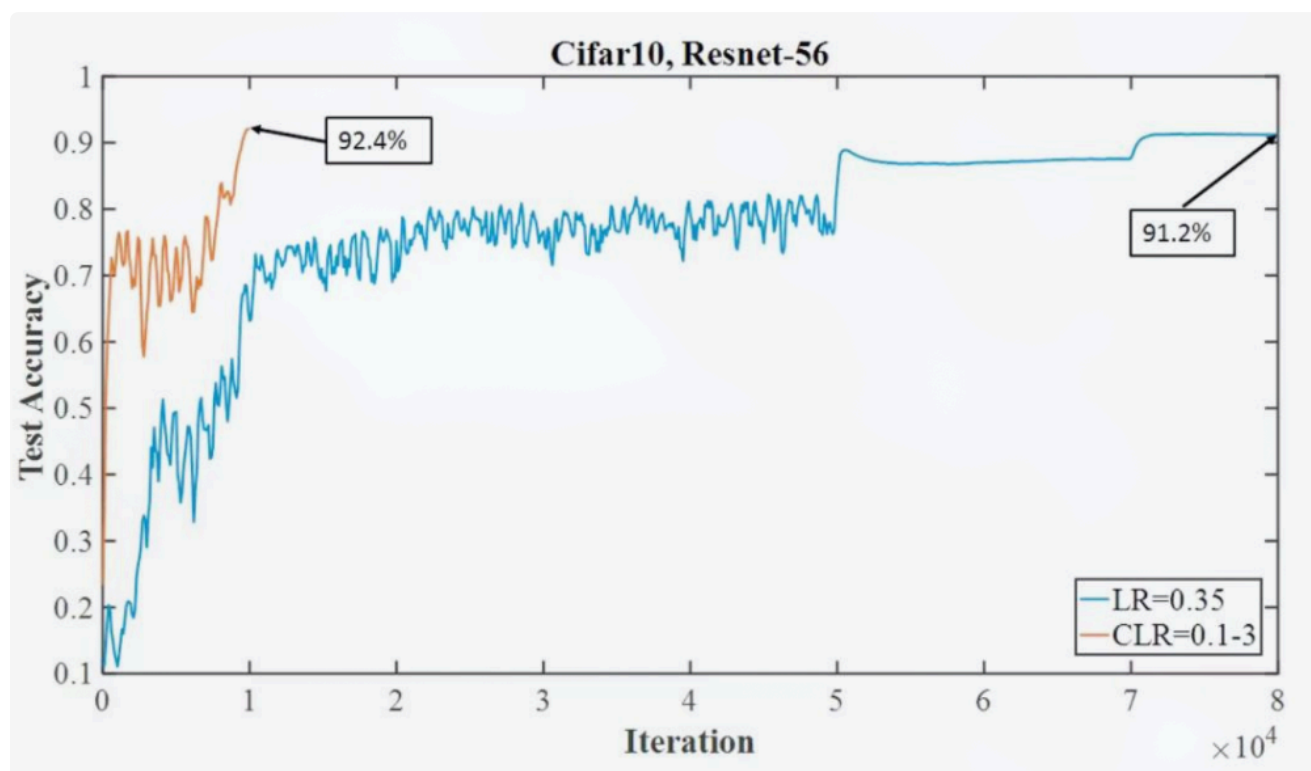
The paper uses the following policies for *learning rate* and *momentum*:



NOTE

It's been shown that **momentum** is **counterproductive** at high learning rates: thus, momentum is **annealed** in the **opposite way** compared to the learning rate

The above mentioned paper has also showed how **much faster** the **superconvergence approach** is when compared to a *fixed LR approach*:



OTHER APPROACHES

In addition to momentum, there are **other rules** that can be used to **update the weights** in a way that **accelerates convergence** of the gradient descent algorithm.

Some examples are:

- *Nesterov Accelerate Gradient (NAG)*.
- *Adaptive Gradient (Adagrad)*.
- **Adadelta**.
- **RMSProp**.
- **Adam**.

Adam is considered to be the **state of the art**, but **doesn't always deliver** better results than the classic SGD + Momentum approach.

Adam is an **optimiser** (introduced in 2014) that computes **adaptive learning rates for each parameter** by combining momentum and other ideas.

More precisely, Adam defines **two moving averages** (m_t , the *first moment*, and u_t , the *second moment*) that are then used to **update the weights**:

$$m_t = \rho_1 m_{t-1} + (1 - \rho_1) g_t$$

$$u_t = \rho_2 u_{t-1} + (1 - \rho_2) g_t^2$$

Where g_t is the value of ∇J at step t and the square on g_t is *component-wise*.

The coefficients ρ_1 and ρ_2 are **hyperparameters** representing the **decay rate** for the moments (usually set to 0.9 and 0.999 respectively).

The moments are initialised to 0:

$$m_0 = 0 \quad u_0 = 0$$

In order to take into account the fact that **value of moving averages** will be **very small due to the zero initialisation**, the authors of Adam defined **bias-corrected versions**:

$$\hat{m}_t = \frac{m_t}{1 - \rho_1}$$

$$\hat{u}_t = \frac{u_t}{1 - \rho_2}$$

Finally, the **update for each weight** of the network is computed as follows:

$$w_{ij}^t = w_{ij}^{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{u}_t} + \varepsilon}$$

Where η is the learning rate and ε is a **very small positive number** to **prevent divisions by zero**, usually set to 10^{-8} .

OVERFITTING AND DROPOUT

In > [0.3 - STATISTICS REVISION > BIAS-VARIANCE TRADEOFF](#) we discussed a key problem in machine learning known as **bias-variance trade-off**.

In particular, **linear models** show **high bias** because they *can only represent a small class of functions*. However, these models also have **low variance**: they give similar results across different random samples of the data.

On the contrary, **deep neural networks** inhabit the *opposite end* of the *bias-variance* spectrum. Because they are *much more complex*, they are **not confined to looking at each feature individually** and can instead **learn interactions among groups of features**.

This translates to an interesting fact:

BIAS-VARIANCE CONSEQUENCE

Linear models **overfit** when presented with **more features than examples**. Conversely, **deep neural networks** tend to **overfit** when faced with **more examples than features**.

In 2017, a group of researchers demonstrated the extreme flexibility of these networks by training DNNs on **randomly-labeled images**.

Despite the **absence of a true pattern** linking the inputs to the outputs, they found that the neural network optimised by SGD could **label every image in the training set perfectly**.

Notice that this means that if **labels are assigned uniformly at random** and there are, say, 10 classes, then **no classifier can do better than 10% accuracy** on *holdout data*.

This raises the question:

QUESTION

If these models are **so complex** and so expressive that they **can overfit this badly**, then **when should we expect them not to overfit?**

The mathematical foundations for the puzzling generalisation properties of deep neural networks remain open research questions.

DROPOUT

The **key expectation** we have from a **good predictive model** is **good performance on unseen data**.

Classical generalisation theory suggests that, to close the gap between train and test performance, we should aim for a **simple model**.

Simplicity may come in the form of a **small number of dimensions**; but another useful notion of simplicity is **smoothness**, that is, the function should **not be sensitive to small changes in its inputs**.

For instance, when classifying images, we would expect that **adding random noise to the pixels should be harmless**.

In 1995, *Christopher Bishop* formalised this idea when he proved that *training input noise is equivalent to Tikhonov regularisation*. This work drew a clear mathematical connection between the requirement that a function be **smooth** and the requirement that it be **resilient to perturbations** in the input.

A clever idea to apply this idea to the internal layers of a network was developed by *Srivastava et al.* in 2014.

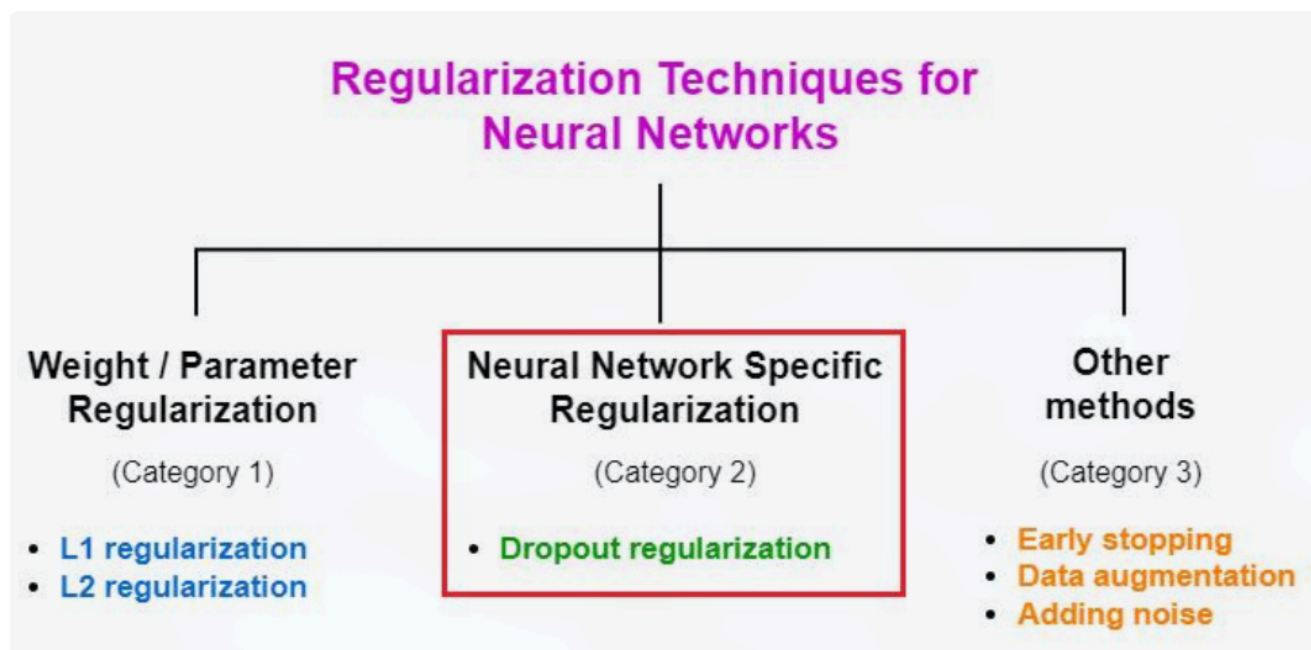
Namely, they proposed to **inject noise onto each layer** of the network in the *forward propagation step* during training and observed that this **enforces smoothness just on the input-output mapping**.

This idea is known as **dropout**:

📖 DROPOUT

Throughout **training**, on *each iteration*, dropout consists of **zeroing out some fraction of the nodes** in **each layer** before calculating the subsequent layer.

Note that dropout falls in the category of **regularisation techniques**, among the ones presented in > [1.0 - NEURAL NETWORKS INTRODUCTION](#) > [LOSS REGULARISATION](#) and others:



The key challenge then is **how to inject this noise in an unbiased manner** so that the **expected value of each layer**, while fixing the others, **remains equal** to the value it would have **outputted without noise**.

In standard dropout regularisation, one "debiases" each layer by **normalising** by the **fraction of nodes that were retained** (not dropped out).

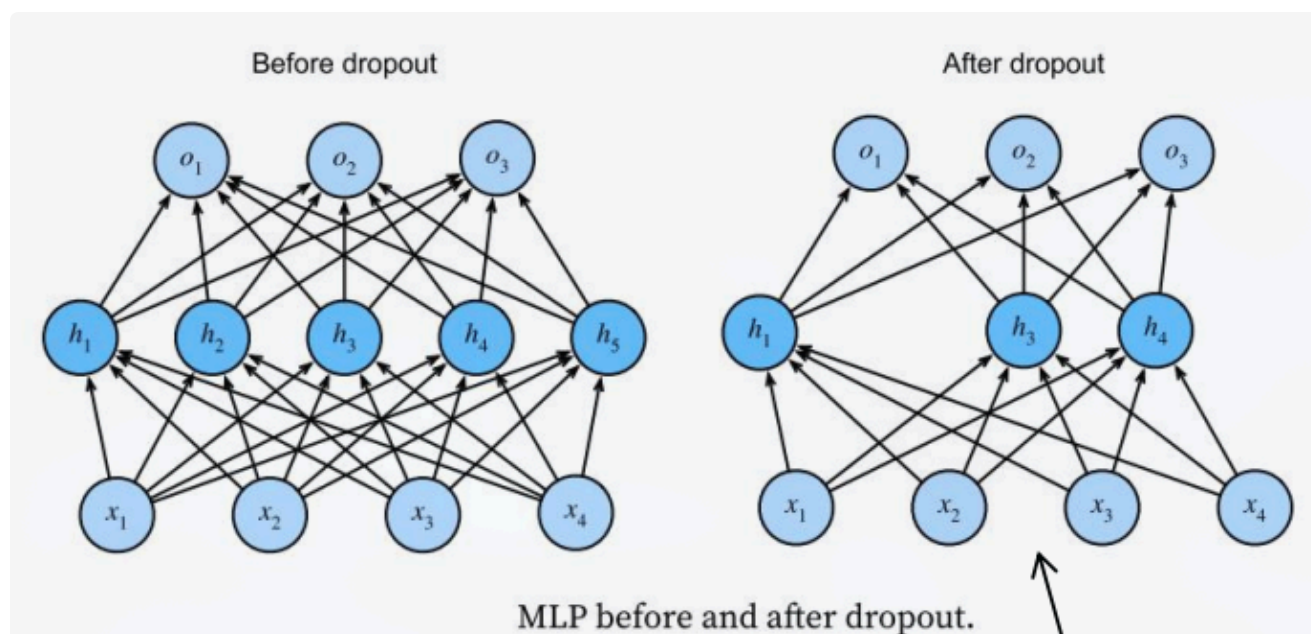
In other words, with *dropout probability* p , each intermediate **activation** h is **replaced by a random variable** h' defined as follows:

$$h' = \begin{cases} 0 & \text{with probability } p \\ \frac{h}{1-p} & \text{otherwise} \end{cases}$$

Thus, we have:

$$\mathbb{E}[h'] = p \cdot 0 + (1-p) \frac{h}{1-p} = h$$

Just as we wanted.



🔗 DROPOUT VISUALISATION

Notice that when *zeroing out each unit with probability* p , the **result** can be viewed as a **network containing only a subset of the original neurons**.

In the example figure above, h_2 and h_5 are removed, and their **respective gradient vanishes during backpropagation**.

This way, the calculation of the **output layer cannot be overly dependent on any one element** of $h_1, \dots, h_5$.

We can imagine that **if neurons are randomly dropped out** of the network during training, then **other neurons will have to step in** and handle the representation required to **make predictions for the missing neurons**.

This is believed to result in **multiple independent internal representations** being learned by the network: the effect is that the network becomes **less sensitive to the specific weights** of neurons.

This in turn results in a network that is capable of **better generalisation** and is **less likely to overfit** the training data.

NOTE

Typically, **dropout is disabled at test time**.

However, there are some **exceptions**: researchers may use dropout at test time as **heuristic for estimating the uncertainty of a network predictions**.

In fact, **if predictions are agree across many different dropout masks**, then we might say that the **network is more confident**.

DATA AUGMENTATION

To help **reduce chances of overfitting** when training a machine learning model, we can employ **data augmentation techniques**.

DATA AUGMENTATION

Data augmentation techniques are used to **increase the amount of data** by **adding slightly modified copies of already existing training patterns** or **creating "synthetic" data** from them.

NOTE

Notice that data augmentation techniques are **problem-specific**: for instance, standard approaches used in *computer vision* for image augmentation *deteriorate the performance in audio classification*.

Since *spectrograms* have *time and frequency* on their axes, standard image augmentation techniques, e.g. reflection, will **hardly make sense**.

Consider this example: if a *bird's hum* is characterised by an increasing pitch over the its duration, a reflection would represent a *completely different sound* where the pitch decreases over its duration.

Here, we will be mainly discussing **image augmentation techniques**, since image classification is amongst the *harder classification problems*.

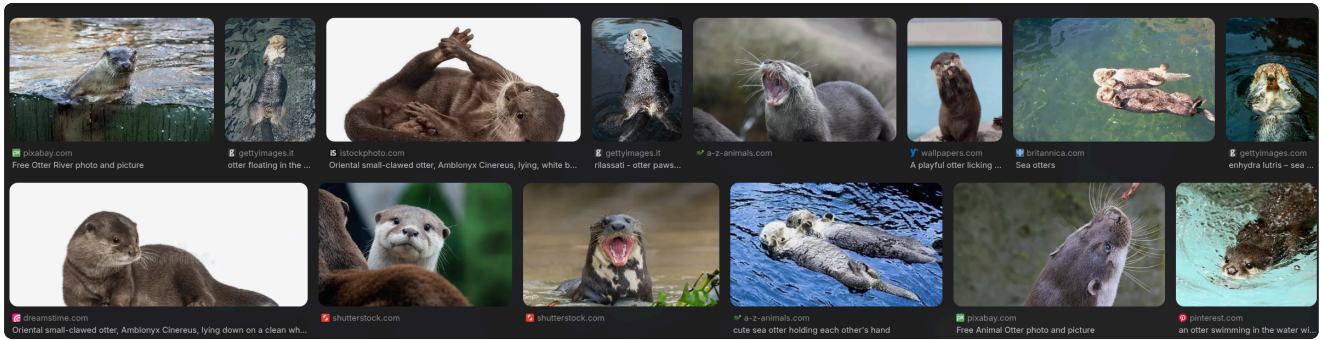
Keep in mind that the motivations behind the need for data augmentation apply to all contexts, but remember the note above and *do not employ these methods to other domains*.

MOTIVATION, INTRA-CLASS VARIATIONS AND GENERALISATION

Deep neural networks must be **invariant to a wide variety of inputs**.

That is, we want our model to **not be affected by small variations in the inputs**.

Take the following images as an example:



Imagine we are training a model to **recognise animals**: all the images above are pictures of *otters*, but they **vary in the animal's position**. These variations are known as *intra-class variations*:

📖 INTRA-CLASS VARIATIONS

Small **variations** in the **data** within a **certain class**.

Notice that other kinds of *intra-class variations* that exist in images are appearance, lighting, orientation etc.

This **large amount** of variations makes **image classification** particularly **challenging**.

To train a model that **can recognise these variations** we need to **train it more data**, but this leads to a practical problem: **data is often limited**.

Thus, the goal of data augmentation is the following:

💡 MOTIVATION

Create "**fake**" data from the *available existing patterns* and **add it to the training set** to achieve the **desired variation** in **training data**.

Notice that this process is **often "on the fly"**: augmented images are **generated using random transformations** from each image in the batch at the beginning of every iteration, but they **are not stored**.

It's also crucial to guarantee the **preservation of semantics**; that is, augmented data **should not change the class** of the original pattern.

GEOMETRIC TRANSFORMATIONS

Even **simple geometric operations** often **greatly improve generalisation**.

Some examples are:

📄 IMAGE CROPPING

- **Randomly crop** and *re-scale* images.

- Do not crop small regions, as *semantics could be lost*.

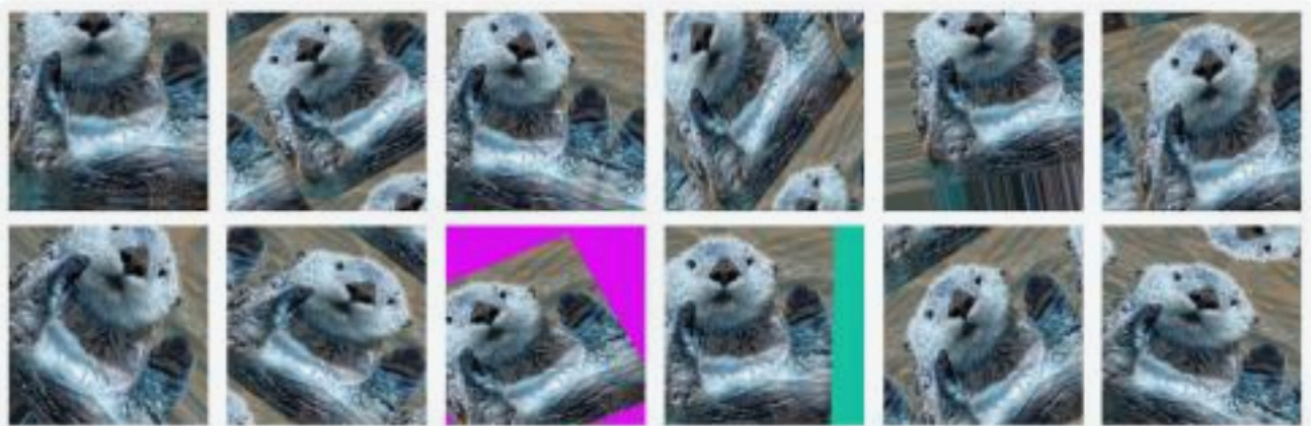
📄 IMAGE FLIPPING

- **Flip** the original images **horizontally** with **some given probability** p .
- **Flip** the original images **vertically** with some **other probability** p' .

NOTE This can't be naively applied to all contexts. Think of *street-related images*: horizontal flipping can **alter the notions of traffic lanes**, leading to *semantic loss*.

📄 AFFINE TRANSFORMATION

- Apply a **single affine/linear transformation** to the 2D image space (i.e. multiply the image matrix by the transformation matrix. Examples are scaling, rotation, shear.).
- Fill **blank spaces** with strategies like *constant colour*, *expand colours*, *replicate colours*.



📄 PIECEWISE AFFINE TRANSFORMATION

- Similar to *affine transformation*, though now a **single transformation is not applied to the whole image**.
- The image is perceived as an underlying **grid**, where each grid undergoes an **independent transformation**.



📋 PERSPECTIVE TRANSFORMATION

- Transformations with **more degrees of freedom**.
- The perspective effect causes **certain regions** to be **enlarged** and others to be **squeezed**.



LOCAL FILTERS

Other commonly employed transformations rely on image **blurring/sharpening** and **edge detection**.

📋 GAUSSIAN BLUR

- Applies a **normally distributed blur filter** onto the given image with a σ chosen at random from a pre-specified interval.
- Helps the model to **recognise objects at different resolutions**, making it more robust to *camera-induced* variations in captured images.



📋 IMAGE SHARPENING

- Does the **opposite of introducing blur**. The resulting effect is a **sharper image**.

📋 EDGE DETECTION

- Uses the **edge-detected** version of a given image.
- It's important to verify that the **detected edges do not change the class** of the objects.



ADDING NOISE

It's been observed that deep neural networks are **highly sensitive to noisy images**. Thus, it's very likely that *adding even a small amount of noise in an image* would lead to *drastic reductions in performance*, even though the *human eye would be mostly undisturbed*.

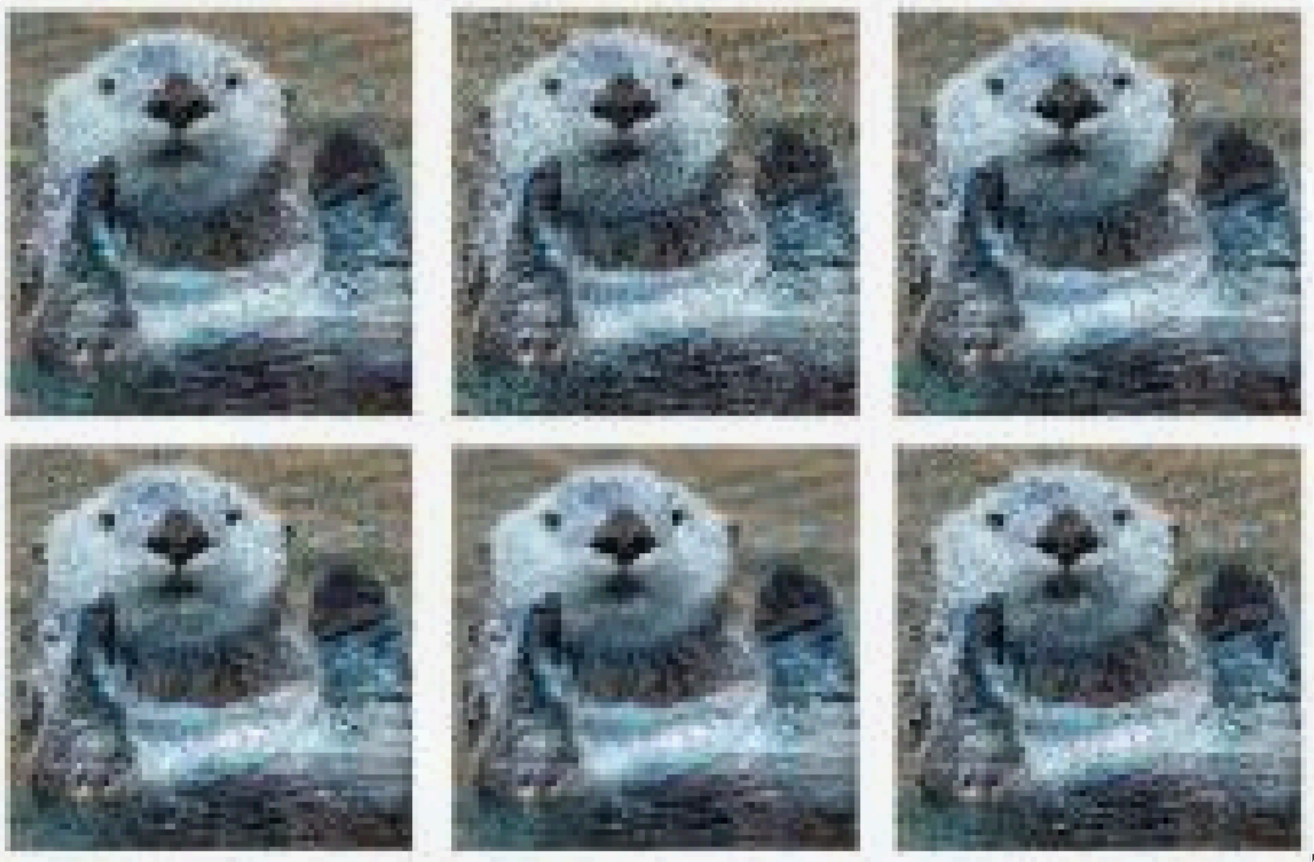
Adding noise is therefore **crucial to help the model generalise better**.

The most commonly used technique is **Gaussian noise**:

📋 GAUSSIAN NOISE

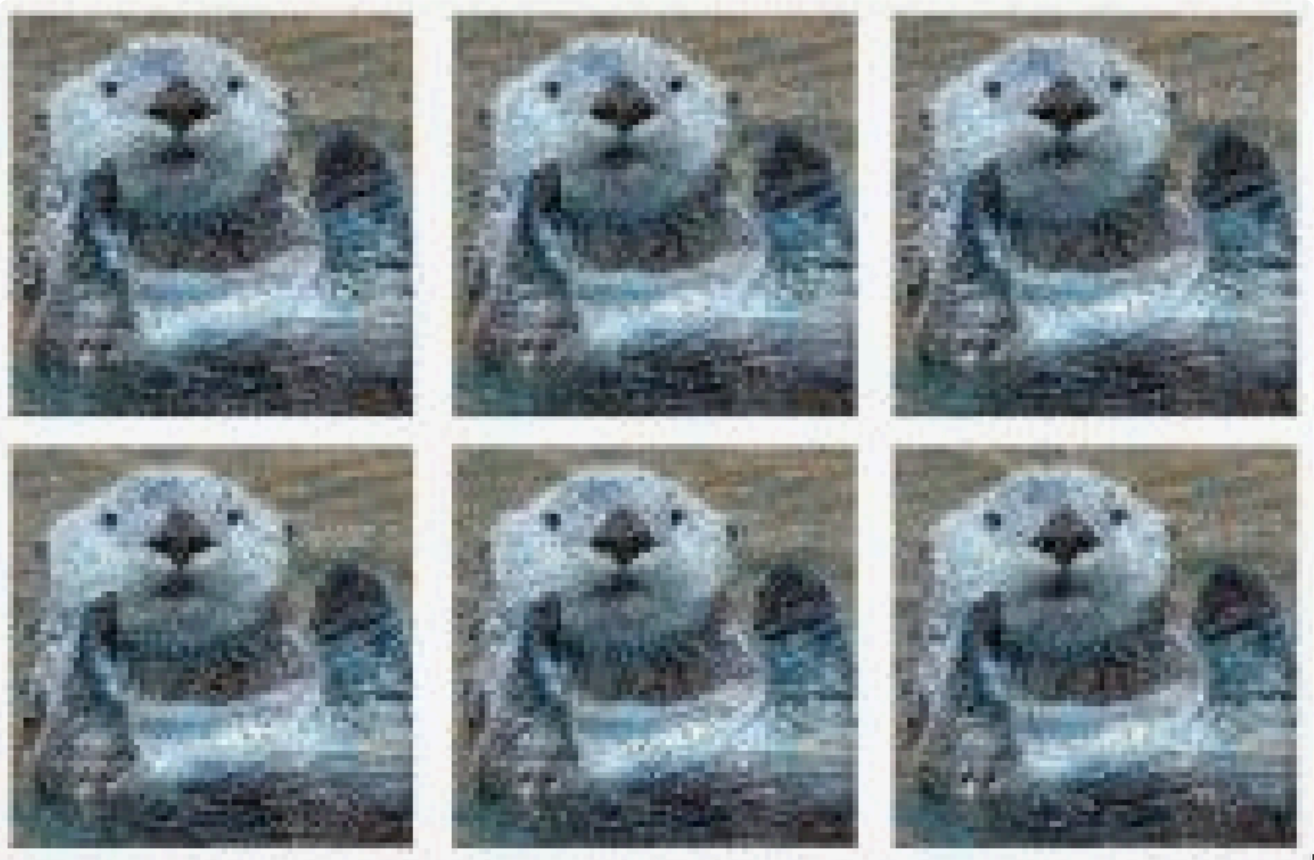
Add a fixed **Gaussian noise** (this simply means *adding normally distributed values*) to **each pixel in the image**.

The additive noise is **chosen randomly for different images** in the data set.



SALT AND PEPPER NOISE

Each pixel in the image is **can be turned either black or white** depending on a specific probability.



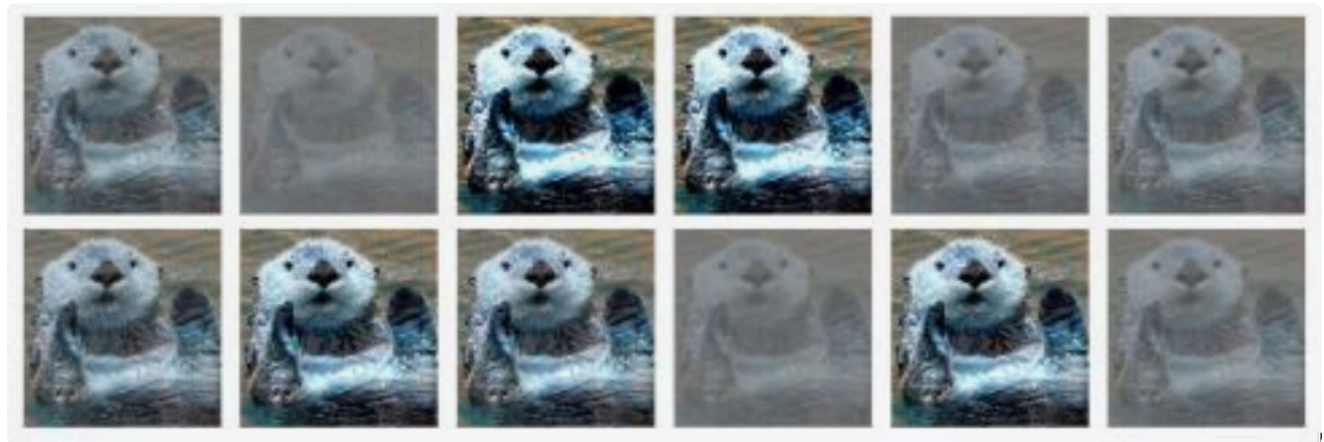
OTHER TRANSFORMATIONS

Colour transformations are also extremely useful, as cameras produce different **colour spectra** depending on the **type of sensors** used, as well as the *white-balancing* performed.

In any case, the *main reason* these transformations are applied is to help the model **generalise to different lighting conditions**. In fact, the captured colours of a scene vary dramatically **depending on the time of the day**.

📋 CONTRAST

Change the **contrast** of an image to either become **faint** or to have **stronger contrast**.



📋 BRIGHTNESS

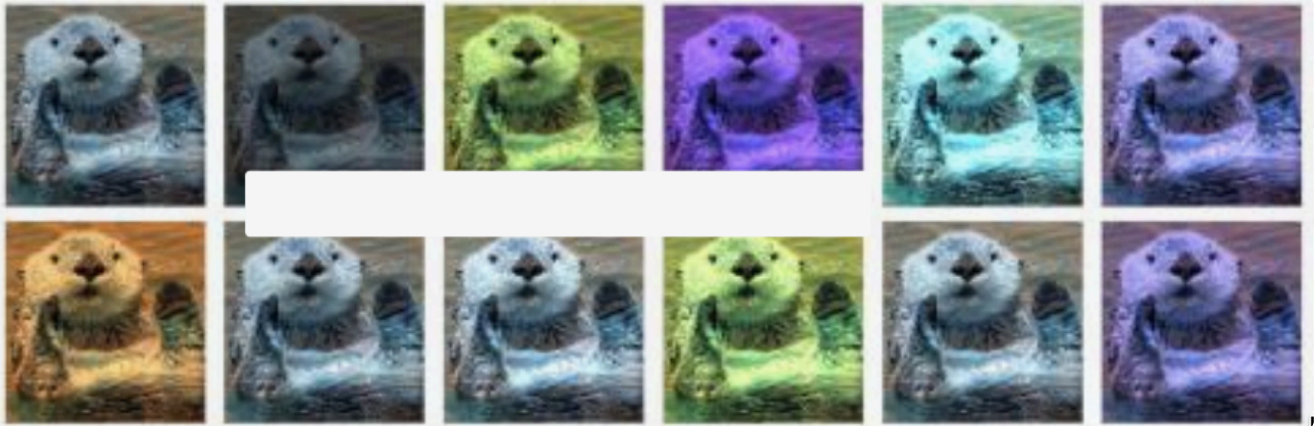
Tweak the **image brightness** to make it either **darker** or **brighter**.



📋 BRIGHTNESS PER CHANNEL

Instead of *changing the brightness for the whole image*, we can instead **tweak for each specific channel** (e.g. the red, green and blue channels in RGB)

images).



Furthermore, even though **weather effects** might sound complex, in reality they are relatively **easy to compute** and can be **done on the fly**.

SNOW

Introduce a **snow-like** effect to the image.



CLOUDS

Introduce an effect that **resembles clouds**.



📄 FOG

Introduces a **fog effect** on the image.



COMBINING TECHNIQUES

In practice, **all the techniques above** are combined and then **randomly applied** on the fly to **different images drawn at random** from within the mini-batch (e.g. we can *rotate an image* and then *apply the fog filter*).

✎ IMPORTANT REMARKS

- When **comparing two networks** we must make sure to use the same **augmentation techniques** for **both**, as data augmentation can *significantly improve performance* and will certainly drive us to *wrong conclusions* if different augmentation strategies are employed for the two models being compared.
- Data augmentation must be considered **part of the network design**.
- It is important to specify the **right distribution of transformations**: not all of them may be useful to a particular task, therefore the **helpful ones** and **their strength** are **hyperparameters** that must be **chosen carefully**.